

Ordenação: algoritmos elementares



“Primeiro, coloque os números em ordem crescente.
Depois decidiremos o que fazer.”
— *estratégia algorítmica*

Este capítulo trata do seguinte problema fundamental: *Permutar* (ou seja, rearranjar) os elementos de um *vetor* $v[0..n-1]$ de tal modo que eles fiquem em *ordem crescente*, isto é, de tal forma que tenhamos $v[0] \leq v[1] \leq \dots \leq v[n-1]$.

0	1	2	3	4	5	6	7	8	9	10
111	999	222	999	333	888	444	777	555	666	555

111	222	333	444	555	555	666	777	888	999	999
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Discutiremos aqui dois algoritmos simples para o [problema](#). Outros capítulos descreverão algoritmos mais sofisticados e bem mais rápidos.

Embora o problema tenha sido apresentado em termos da ordenação de um vetor, ele faz sentido para qualquer estrutura linear, como uma [lista encadeada](#), por exemplo.

Sumário:

- [Ordenação por inserção](#)
- [Ordenação por seleção](#)
- [Estabilidade da ordenação](#)

Exercício 1

1. **IMPORTANTE.** Escreva uma função que verifique se um vetor $v[0..n-1]$ está em ordem crescente. (Este exercício põe em prática a estratégia de *escrever os testes* antes de inventar algoritmos para o problema.)
2. Critique o código da seguinte função, que promete decidir se o vetor $v[0..n-1]$ está em ordem crescente.

```
int verifica (int v[], int n) {
    if (n > 1)
        for (int i = 1; i < n; i++)
            if (v[i-1] > v[i]) return 0;
    return 1; }
```

3. Critique o código da seguinte função, que promete decidir se o vetor $v[0..n-1]$ está em ordem crescente.

```
int verifica (int v[], int n) {
    int sim;
    for (int i = 1; i < n; i++)
        if (v[i-1] <= v[i]) sim = 1;
        else {
            sim = 0;
            break; }
    return sim; }
```

4. Escreva uma função que rearranje um vetor $v[0..n-1]$ de modo que ele fique em ordem [estritamente crescente](#).

5. Escreva uma função que receba um inteiro x e um vetor $v[0..n-1]$ de inteiros em ordem crescente e [insira](#) x no vetor de modo a manter a ordem crescente.

Ordenação por inserção

O algoritmo de ordenação por inserção, ou algoritmo Insertionsort, é frequentemente usado para colocar em ordem um baralho de cartas (veja página [aula da Khan Academy](#)):

```
// Esta função rearranja o vetor v[0..n-1]
// em ordem crescente.

void
insertionsort (int n, int v[])
{
    for (int j = 1; j < n; ++j) {
        int x = v[j];
        int i;
        for (i = j-1; i >= 0 && v[i] > x; --i)
            v[i+1] = v[i];
        v[i+1] = x;
    }
}
```

(Compare o for interno com o [algoritmo de inserção discutido no capítulo Vetores](#).) Para entender o funcionamento do algoritmo, basta observar que no início de cada repetição do for externo, imediatamente antes da comparação “ $j < n$ ”,

1. o vetor $v[0..n-1]$ é uma permutação do vetor original e
2. o vetor $v[0..j-1]$ está em ordem crescente.

Essas condições [invariantes](#) são trivialmente verdadeiras no início da primeira iteração, quando j vale 1. No início da última iteração, j vale n e portanto o vetor $v[0..n-1]$ está em ordem, como desejado. (Note que a última iteração é abortada logo no início, pois a condição “ $j < n$ ” é falsa.)

0	crescente			j-1	j						n-1
444	555	555	666	777	222	999	222	999	222	999	

Desempenho do algoritmo. Quantas vezes a função insertionsort compara x com um elemento do vetor? Mais exatamente, quantas vezes, no máximo, a função insertionsort executa a comparação “ $v[i] > x$ ”? Esse número está diretamente relacionado com a sequência de valores de i ao longo da execução da função. Para cada valor de j , a variável i assume, no [pior caso](#), os valores $j-1, \dots, 0$. A seguinte tabela mostra esses valores explicitamente:

j	i	
1	0	1
2	1 0	2

3	2	1	0		3
.	.	.	.		.
n-1	n-2	n-3	..	1	0
					n-1

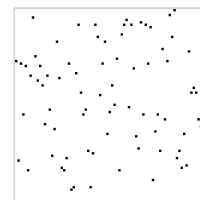
A terceira coluna da tabela dá o número de diferentes valores de i na mesma linha da segunda coluna. Portanto, o número de execuções da comparação " $v[i] > x$ " é, no pior caso, igual à soma da terceira coluna. Essa soma é $n(n-1)/2$, ou seja, um pouco menos que a metade de

$$n^2.$$

Agora considere o *tempo* que a função `insertionsort` consome para fazer o serviço. Esse tempo é *proporcional ao número de execuções da comparação* " $v[i] > x$ " (pense!). Logo, o consumo de tempo da função cresce, no pior caso, como o quadrado do tamanho do vetor. Se um vetor de tamanho N consome T segundos então um vetor de tamanho $2N$ consumirá $4T$ segundos e um vetor de tamanho $10N$ consumirá $100T$ segundos!

Essa análise mostra que o algoritmo Insertionsort é lento demais para ordenar vetores grandes. Graças à sua simplicidade, entretanto, o algoritmo é muito útil no caso de vetores pequenos. Além disso, no [melhor caso](#) (por exemplo, se o vetor já está "quase ordenados"), o desempenho do algoritmo é muito bom: o número de comparações não passa de n e portanto o consumo de tempo é proporcional a n .

Animações. A animação à direita (copiada da [Wikimedia](#)) mostra a ordenação por inserção de um vetor $v[0..79]$ de números positivos. (Veja uma [versão mais lenta](#) da animação.) Cada elemento $v[i]$ do vetor é representado pelo ponto de coordenadas $(i, v[i])$. Há várias outras animações interessantes na rede WWW:



- [Insertion sort](#), na Wikipedia.
- [Insert-sort com dança folclórica Romena](#), Universidade Sapiientia (Romênia).

Exercícios 2

1. Escreva uma versão *recursiva* do algoritmo de ordenação por inserção.
2. Na função [insertionsort](#), troque a comparação " $v[i] > x$ " por " $v[i] >= x$ ". A nova função continua correta?
3. Que acontece se trocarmos " $\text{for } (j = 1$ " por " $\text{for } (j = 0$ " no código da função [insertionsort](#)? Que acontece se trocarmos " $v[i+1] = x$ " por " $v[i] = x$ "?
4. Critique a seguinte implementação do algoritmo de ordenação por inserção:

```
for (int j = 1; j < n; ++j) {
    int x = v[j];
    for (int i = j-1; i >= 0 && v[i] > x; --i) {
        v[i+1] = v[i];
        v[i] = x; } }
```

5. Critique a seguinte implementação do algoritmo de ordenação por inserção:

```
int i, j, x;
for (j = 1; j < n; ++j) {
    for (i = j-1; i >= 0 && v[i] > v[i+1]; --i) {
        x = v[i]; v[i] = v[i+1]; v[i+1] = x; } }
```

6. A seguinte implementação do algoritmo de ordenação por inserção está correta?

```
for (int j = 1; j < n; ++j) {
    int x = v[j];
    int i = j - 1;
    while (i >= 0 && v[i] > x) {
        v[i+1] = v[i];
        --i; }
    v[i+1] = x; }
```

7. **TESTE DE CORREÇÃO.** Escreva um programa para testar, experimentalmente, a correção de sua implementação do algoritmo Insertionsort. Faça testes com vetores inteiros [aleatórios](#) de vários tamanhos. Depois de cada teste, seu programa deve [verificar](#) se o resultado está, de fato, em ordem crescente.
8. **IMPORTANTE.** Escreva uma versão do algoritmo Insertionsort que reorganize em ordem crescente um vetor $v[p..r]$ e tenha o seguinte invariante: no início de cada iteração, o vetor $v[k+1..r]$ é crescente.
9. **BUSCA BINÁRIA.** O papel do `for` interno na função [insertionsort](#) é encontrar o ponto onde $v[j]$ deve ser inserido em $v[0..j-1]$. Considere fazer isso com uma [busca binária](#). Analise o resultado.
10. **PIOR CASO.** Descreva e analise uma [instância](#) de pior caso para o algoritmo Insertionsort, ou seja, um vetor $v[0..n-1]$ que leva o algoritmo a executar o maior número possível de comparações.
11. **MELHOR CASO.** Descreva e analise uma instância de melhor caso para o algoritmo Insertionsort, ou seja, um vetor $v[0..n-1]$ que leva o algoritmo a executar o menor número possível de comparações.
12. Seja T a função sobre inteiros positivos definida pela fórmula $T(N) = cN^2$, onde c é uma constante. Prove que $T(2N) = 4T(N)$ para todo N .
13. ★ **MOVIMENTAÇÃO DE DADOS.** Quantas vezes, no pior caso, o algoritmo Insertionsort copia um elemento do vetor de um lugar para outro? Quantas vezes isso ocorre no melhor caso?
14. Escreva uma função com protótipo `insertionsort1(int v[], int n)` que reorganize um vetor $v[1..n]$ (note os índices!) em ordem crescente. (Basta invocar `insertionsort` da maneira correta.)
15. **ORDEM DECRESCENTE.** Escreva uma função que permuta os elementos de um vetor $v[0..n-1]$ de modo que eles fiquem em ordem decrescente. Inspire-se no algoritmo de ordenação por injeção.
16. ★ **TESTE DE DESEMPENHO.** Escreva um programa para cronometrar sua implementação do algoritmo Insertionsort. Eis uma sugestão sobre como isso pode ser feito. Para um valor fixo de n , um *experimento* consiste no seguinte: gere um vetor inteiro [aleatório](#), submeta-o à sua implementação, e use a função `clock` da [biblioteca time](#) para medir o tempo que sua implementação consome. Para cada valor de n , faça alguns (uma dúzia, digamos) experimentos e tome a média das medidas de tempo. Repita o processo todo para uma sequência de valores de n . (Você pode tomar a sequência $2^8, 2^9, \dots, 2^{19}, 2^{20}$, por exemplo.) Apresente os resultados numa tabela cada uma de cujas linhas corresponde a um valor de n . A tabela pode ter três colunas: na coluna 1, o valor de n ; na coluna 2, a média dos consumos de tempos para aquele valor de n ; na coluna 3, o tempo médio da coluna 2 dividido por n^2 . Com essa tabela, será fácil comparar a prática com a teoria.

Ordenação por seleção

Esta seção trata do algoritmo de ordenação por seleção, ou algoritmo Selectionsort. Ele usa a seguinte estratégia: seleciona o menor elemento do vetor, depois o segundo menor, depois o terceiro menor, e assim por diante.

```
// Esta função reorganiza o vetor v[0..n-1]
// em ordem crescente.

void
selectionsort (int n, int v[])
{
    for (int i = 0; i < n-1; ++i) {
        int min = i;
        for (int j = i+1; j < n; ++j)
            if (v[j] < v[min]) min = j;
        int x = v[i]; v[i] = v[min]; v[min] = x;
    }
}
```

Para entender por que o algoritmo está correto, basta observar que no início de cada repetição do `for` externo, imediatamente antes da comparação “ $i < n-1$ ”, valem os seguintes [invariantes](#):

1. o vetor $v[0..n-1]$ é uma permutação do vetor original,
2. $v[0..i-1]$ está em ordem crescente e

$$3. v[i-1] \leq v[i..n-1].$$

A tradução do terceiro invariante para linguagem humana é a seguinte: $v[0..i-1]$ contém todos os elementos “pequenos” do vetor original e $v[i..n-1]$ contém todos os elementos “grandes”.

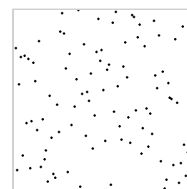
0	crescente				i-1	i					n-1
110	120	120	130	140	999	666	999	666	999	666	
pequenos					grandes						

Os três invariantes garantem que no início de cada iteração $v[0], \dots, v[i-1]$ já estão em suas posições definitivas. No início da última iteração, o vetor está em ordem crescente, como desejado.

Desempenho do algoritmo. Tal como o de ordenação por inserção, o algoritmo de ordenação por seleção faz cerca de $n^2/2$ comparações entre elementos do vetor no [pior caso](#).

Diferentemente do algoritmo Insertionsort, entretanto, o algoritmo Selectionsort também faz cerca de $n^2/2$ comparações no [melhor caso](#) (por exemplo, quando o vetor já está “quase ordenado”). Assim, o consumo de tempo do algoritmo é sempre proporcional a n^2 . Em vista dessa análise (e de outras observações que faremos nos exercícios) o algoritmo Insertionsort é preferível ao de seleção.

Animações. A animação à direita (copiada da [Wikipedia](#)) mostra a ordenação por seleção de um vetor $v[0..79]$ de números positivos. (A velocidade da animação não representa a velocidade do algoritmo. Veja uma [versão mais lenta](#) da animação.) Cada elemento $v[i]$ do vetor é representado pelo ponto de coordenadas $(i, v[i])$. Há várias outras animações interessantes na rede WWW:



- [Selection sort](#), na Wikipedia.
- [Select-sort com dança cigana](#), Universidade Sapientia (Romênia).

Exercícios 3

1. Escreva uma versão *recursiva* do algoritmo de ordenação por seleção.
2. Na função [selectionsort](#), que acontece se trocarmos “for ($i = 0$)” por “for ($i = 1$)”? Que acontece se trocarmos “for ($i = 0$; $i < n-1$)” por “for ($i = 0$; $i < n$)”?
3. Na função [selectionsort](#), troque a comparação “ $v[j] < v[\min]$ ” por “ $v[j] \leq v[\min]$ ”. A nova função continua correta?
4. **TESTE DE CORREÇÃO.** Escreva um programa para testar, experimentalmente, a correção de sua implementação do algoritmo Selectionsort. (Veja [exercício análogo para o Insertionsort](#).)
5. **PIOR CASO.** Descreva e analise uma [instância](#) de pior caso para o algoritmo Selectionsort, ou seja, um vetor $v[0..n-1]$ que leva o algoritmo a executar o maior número possível de comparações.
6. **MELHOR CASO.** Descreva e analise uma instância de melhor caso para o algoritmo Selectionsort, ou seja, um vetor $v[0..n-1]$ que leva o algoritmo a executar o menor número possível de comparações.
7. ★ **MOVIMENTAÇÃO DE DADOS.** Quantas vezes, no pior caso, o algoritmo Selectionsort copia um elemento do vetor de um lugar para outro? Quantas vezes isso ocorre no melhor caso?
8. **ORDENAÇÃO DECRESCENTE.** Escreva uma função que permute os elementos de um vetor inteiro $v[0..n-1]$ de modo que eles fiquem em ordem decrescente. Inspire-se no algoritmo Selectionsort.
9. ★ **TESTE DE DESEMPENHO.** Escreva um programa para cronometrar sua implementação do Selectionsort. (Veja o [exercício análogo para o Insertionsort](#).)

Exercícios 4

1. ORDENAÇÃO DE STRINGS. Escreva uma função que coloque em [ordem lexicográfica](#) um vetor de [strings ASCII](#). Use o algoritmo Insertionsort.
2. ORDENAÇÃO DAS LINHAS DE UM ARQUIVO. Escreva uma função que rearranje as linhas de um [arquivo ASCII](#) em [ordem lexicográfica](#). (Veja a descrição do utilitário [sort](#).)
3. ORDENAÇÃO DE STRUCTS. Suponha que cada elemento de um vetor é um [registro](#) com dois campos: um é um inteiro e outro uma [string ASCII](#):

```
struct registro {int aa; char *bb;;};
```

(O campo `aa` pode ser o número de uma peça no estoque de uma loja e `bb` o nome da peça.) Escreva uma função que rearranje o vetor de modo que os campos `aa` fiquem em ordem crescente. Escreva outra função que rearranje o vetor de modo que os campos `bb` fiquem em [ordem lexicográfica](#).

4. DESAFIO. Invente um algoritmo de ordenação que seja *mais rápido* que o de inserção e o de seleção.
5. EMBARALHAMENTO ALEATÓRIO? Considere a seguinte antítese do problema de ordenação: fazer um embaralhamento aleatório dos elementos de um vetor $v[0..n-1]$, ou seja, rearranjar os elementos do vetor de modo que todas as [permutações](#) sejam igualmente prováveis. Discuta e critique a seguinte solução (ela foi [usada na distribuição européia do Windows 7 da Microsoft](#)):

```
void insertionsort_al (int n, int v[]) {
    for (int j = 1; j < n; ++j) {
        int x = v[j];
        int i;
        for (i = j-1; i >= 0; --i) {
            if (rand\(\). > RAND\_MAX/2)
                v[i+1] = v[i];
            else break; }
        v[i+1] = x; } }
```

6. ORDENAÇÃO DE LISTA ENCADEADA. Escreva uma função que ordene uma [lista encadeada](#). Inspire-se no algoritmo Insertionsort para vetores. (Sua função precisa devolver alguma coisa?).
7. ORDENAÇÃO DE LISTA ENCADEADA. Escreva um função para ordenar uma lista encadeada. Imite o algoritmo Selectionsort para vetores. (Sua função precisa devolver alguma coisa?).

Exercícios 5: aplicações

Muitos problemas podem ser reduzidos à ordenação de um vetor, ou seja, podem ser resolvidos com o auxílio de um algoritmo de ordenação (não necessariamente um dos algoritmos discutidos neste capítulo).

1. ANAGRAMAS. Uma palavra é *anagrama* de outra se a [sequência](#) de letras de uma é permutação da sequência de letras da outra. Por exemplo, “aberto” é anagrama de “rebato”. Digamos que duas palavras são *equivalentes* se uma é anagrama da outra. Uma *classe de equivalência* de palavras é um conjunto de palavras duas a duas equivalentes. Escreva um programa que receba um [arquivo ASCII](#) contendo palavras (uma por linha) e extraia desse arquivo uma classe de equivalência máxima. Aplique o seu programa a um arquivo como [br-sem-acentos.txt](#) que contém todas as palavras do português brasileiro com sinais diacríticos removidos. (O arquivo é grande; portanto, o seu programa deverá ser muito eficiente.)
2. VALORES DISTINTOS. Dado um vetor $v[0..n-1]$ de números inteiros, determinar quantos números distintos há no vetor (ou seja, encontrar o tamanho do [conjunto](#) de elementos do vetor).
3. MEDIANA. Seja $v[0..n-1]$ um vetor de números inteiros, todos diferentes entre si. A *mediana* do vetor é um elemento do vetor que seja maior que metade dos elementos do vetor e menor que (a outra) metade dos elementos. Mais exatamente, a mediana de $v[0..n-1]$ é um número m que seja igual a algum elemento do vetor e estritamente maior que exatamente $\lfloor (n-1)/2 \rfloor$ elementos do vetor. (Não confunda mediana com média. Por exemplo, a média de 1 2 99 é 51, enquanto a mediana é 2.) Escreva um algoritmo que encontre a mediana de um vetor $v[0..n-1]$ de inteiros distintos dois a dois.

4. ESCALONAMENTO DE TAREFAS. Suponha que n tarefas devem ser processadas em um único processador. Dadas as durações $t_1, \dots, t_n$ das tarefas, em que ordem elas devem ser processadas para minimizar o *tempo médio* de conclusão de uma tarefa?

Estabilidade da ordenação

Um algoritmo de ordenação é *estável* (= *stable*) se não altera a posição relativa dos elementos que têm o mesmo valor. Digamos, por exemplo, que um vetor de [números do tipo double](#) é ordenado considerando apenas a parte inteira dos números (e ignorando a parte fracionária). Se o algoritmo de ordenação for estável, os números que têm a mesma parte inteira continuarão na mesma ordem relativa em que estavam originalmente. Na seguinte figura, a primeira linha mostra o vetor original e a segunda, o vetor ordenado por um algoritmo estável:

44.0	55.1	55.2	33.9	22.9	11.0	22.5	22.6
11.0	22.9	22.5	22.6	33.9	44.0	55.1	55.2

0:00 / 0:52

Eis outro exemplo. Digamos que cada elemento do vetor é um [struct](#) com dois campos: o primeiro contém o nome de uma pessoa e o segundo contém o ano em que a pessoa nasceu. Suponha que o vetor original tem dois “João da Silva”, primeiro o que nasceu em 1990 e depois o que nasceu em 1995. Se o vetor for ordenado por um algoritmo estável com base no primeiro campo, os dois “João da Silva” continuarão na mesma ordem relativa: primeiro o de 1990 e depois o de 1995.

A estabilidade de um algoritmo de ordenação é uma propriedade útil que usaremos em [outro capítulo](#).

Exercícios 6

1. A função `insertionsort` analisada [acima](#) implementa uma ordenação estável?
2. Na função [insertionsort](#), troque a comparação “`v[i] > x`” por “`v[i] >= x`”. A função modificada faz uma ordenação estável?
3. A função `selectionsort` analisada [acima](#) implementa uma ordenação estável?

Veja os verbetes [Sorting algorithm](#), [Insertion sort](#) e [Selection sort](#) na Wikipedia.

Veja o capítulo 11 do [Programming Pearls](#) de Jon Bentley.

Atualizado em 2019-02-01

<https://www.ime.usp.br/~pf/algoritmos/>

© Paulo Feofiloff

[DCC-IME-USP](#)